

Lab 2: Procesamiento de logs y texto en Debian

Objetivos

Practicar:

- `cat`
- `less`
- `head`
- `tail`
- `grep`
- `cut`
- `sort`
- `uniq`
- `wc`
- pipes (`|`)
- análisis básico de logs

Contexto

En Debian Trixie muchos sistemas utilizan `systemd-journald` y puede no existir `rsyslog`.

Los logs pueden consultarse mediante:

`journalctl`

o en archivos dentro de:

`/var/log/`

Parte 1 – Exploración de logs

Ejercicio 1

Listar logs disponibles:

`ls /var/log`

Ejercicio 2

Ver primeras líneas:

```
head /var/log/auth.log
```

Ver últimas:

```
tail /var/log/auth.log
```

Seguir eventos de logueo en tiempo real:

Este ejemplo sirve como un modo muy sencillo de monitorear los usuarios que se están logueando en un equipo sin instalar ninguna aplicación adicional.

Nota sobre archivo auth:

en algunas distribuciones que no instalan el paquete rsyslog puede suceder que no se encuentre el archivo «auth.log». La misma información se puede obtener del demonio ssh mediante el comando

```
journalctl -u ssh
```

Con lo cual se puede utilizar su salida con un pipe para probar todos los comandos de manejos de archivos que estuvimos revisando en la clase ej:

```
journalctl -u ssh | grep -i invalid
```

```
journalctl -u ssh | grep -i invalid | cut -d ' ' -f 8,10
```

...etc...

a) En una consola ejecutar lo siguiente y dejarlo procesando (-f hace que se quede listando en la consola cada nueva línea que se genere.

```
tail -f /var/log/auth.log
```

o

```
journalctl -f -u ssh
```

b) Desde otra consola (Si están en consola de texto pasan de una consola a otra presionando [Ctrl] [F1] , [Ctrl][F2] ...) o loguéandose desde otra pestaña del emulador gráfico de terminal o el putty, sin cerrar la anterior, intentar loguearse varias veces algunas de manera correcta y otras con contraseña o usuario erróneo.

Parte 2 – Buscar intentos fallidos

Ejercicio 3

Buscar intentos fallidos:

```
grep "Failed password" /var/log/auth.log
```

Ejercicio 4

Contar intentos fallidos:

```
grep "Failed password" /var/log/auth.log | wc -l
```

Parte 3 – Trabajar con columnas usando cut

Ejercicio 5

Mostrar sólo las primeras columnas del log:

```
grep "Failed password" /var/log/auth.log | cut -d " " -f1-12
```

Ejercicio 6

Intentar extraer la IP:

```
grep "Failed password" /var/log/auth.log | cut -d " " -f11
```

Preguntas

1. ¿La IP aparece correctamente siempre?
2. ¿Por qué a veces cut falla con espacios?
3. ¿Qué ocurre si hay múltiples espacios consecutivos?

Parte 4 – Limpiar espacios

Nota: En los siguientes ejemplos, vamos a ver que en una consola también podemos escribir varios comandos en varias líneas como si estuviesen en la misma línea. En Bash esto se logra con el caracter "\" que significa: este comando continúa en la línea siguiente. Si quieren también pueden escribir todo el encadenamiento de comando con pipe en la misma línea. Separarlo en varias líneas es solo un tema de claridad de visualización. El caracter "\" no tiene ninguna otra función específica más que cortar un único comando en varias líneas.

Ejercicio 7

Normalizar espacios usando tr:

```
grep "Failed password" /var/log/auth.log | tr -s " "
```

Ahora volver a probar:

```
grep "Failed password" /var/log/auth.log \
```

```
| tr -s " " \
```

```
| cut -d " " -f11
```

Preguntas

1. ¿Qué hace `tr -s`?
2. ¿Por qué ahora funciona mejor `cut`?

Parte 5 – Ordenar y contar IPs

Ejercicio 8

Contar IPs repetidas:

```
grep "Failed password" /var/log/auth.log \  
| tr -s " " \  
| cut -d " " -f11 \  
| sort \  
| uniq -c
```

Ordenar de mayor a menor:

```
grep "Failed password" /var/log/auth.log \  
| tr -s " " \  
| cut -d " " -f11 \  
| sort \  
| uniq -c \  
| sort -nr
```

Parte 6 – Usuarios atacados

Ejercicio 9

Extraer usuarios:

```
grep "Failed password" /var/log/auth.log \  
| tr -s " " \  
| cut -d " " -f9
```

Contarlos:

```
grep "Failed password" /var/log/auth.log \  
|
```

```
| tr -s " " \  
| cut -d " " -f9 \  
| sort \  
| uniq -c \  
| sort -nr
```

Desafío Final

Construir un pipeline que muestre:

- IP
- cantidad de intentos
- ordenados de mayor a menor

Usando únicamente:

grep

tr

cut

sort

uniq

wc

Preguntas conceptuales

1. ¿Qué diferencia hay entre `cat` y `less`?
2. ¿Cuándo conviene usar `tail -f`?
3. ¿Por qué `cut` tiene problemas con espacios variables?
4. ¿Qué ventaja tiene combinar comandos con pipes?
5. ¿Qué función cumplen los logs en Linux?